FACTORYFLY SENTINEL

Human-Guided Physical AI for Active Factory Inspection

TECHNICAL REPORT

AMD AI DevMaster Hackathon 2026 - Track 3
Author: Jaewon Lee
Solo Participant
4 August 2026 | Prototype v7.3.13

Project in one sentence
FactoryFly Sentinel converts repeated human-operated drone inspections into localized visual-change evidence, routes uncertain observations to targeted reinspection, and produces a self-contained report for human review.

Submission disclosure: all demonstration data were self-collected in a private indoor environment. No employer data, employer code, confidential factory information, or proprietary company assets are included.

Executive Summary

Industrial inspection teams can collect large volumes of RGB video, but reviewing every frame and returning to the exact location of an uncertain observation remain costly and error-prone. FactoryFly Sentinel is a human-guided Physical AI prototype that adds persistent spatial memory and an evidence-driven reinspection loop to ordinary drone footage.

The system builds a COLMAP-based 3D baseline from a reference flight, localizes a later inspection inside the same coordinate system, geometrically aligns comparable views, and sends geometry-ready image pairs to an AMD Radeon GPU on Radeon Cloud. A pretrained open-source DINOv2 ViT-S/14 model runs through PyTorch and ROCm to produce relative semantic-change heatmaps and scores.

FactoryFly clears stable observations, records well-supported visual changes, and converts geometrically uncertain high-change evidence into a targeted reinspection mission. The follow-up view is reacquired and analyzed again to determine whether the visual change persists.

Demonstrated end-to-end result
18 analyzed evidence entries | 10 stable observations automatically cleared | 4 confirmed visual findings | 1 targeted reinspection | 0 unresolved findings. The reinspection target was reacquired with good geometry; initial p95 = 0.866 and reinspection p95 = 0.860.

Contents

- 1. Target Application
- 2. System Architecture and Solution Design
- 3. Dataset
- 4. Algorithms and Implementation
- 5. AMD Radeon GPU and ROCm Utilization
- 6. Results and Evaluation
- 7. Innovations and Key Technical Contributions
- 8. Final Deliverables
- 9. Limitations and Future Work
- 10. Team Members and Contributions
- 11. Conclusion

1. Target Application

1.1 Problem

Repeated visual inspection is common in factories, warehouses, utilities, and large facilities. A human operator can fly a drone through an area, but video volume, viewpoint inconsistency, and uncertain observations make evidence review difficult.

1. Video volume grows faster than reviewers can inspect it.
2. A later flight rarely reproduces the same camera pose exactly.
3. When evidence is uncertain, the operator needs actionable guidance about where to return.

Conventional frame-difference methods are fragile under viewpoint, illumination, parallax, motion blur, and temporary occlusion. A useful inspection system therefore needs both spatial context and semantic comparison.

1.2 Proposed Application

- A human operates an RGB drone.
- A baseline flight becomes persistent 3D spatial memory.
- A later inspection is localized inside that memory.
- Comparable views are geometrically refined.
- AMD-accelerated DINOv2 inference generates visual-change evidence.
- Evidence is triaged into stable, confirmed, or reinspection-required states.

- A targeted spatial mission guides the follow-up observation.
- A self-contained evidence report is produced for human review.

Scope boundary
FactoryFly does not diagnose defects, determine safety, or classify operational acceptability. It reports localized visual-change evidence and evidence quality; a human reviewer assigns the final operational disposition.

2. System Architecture and Solution Design

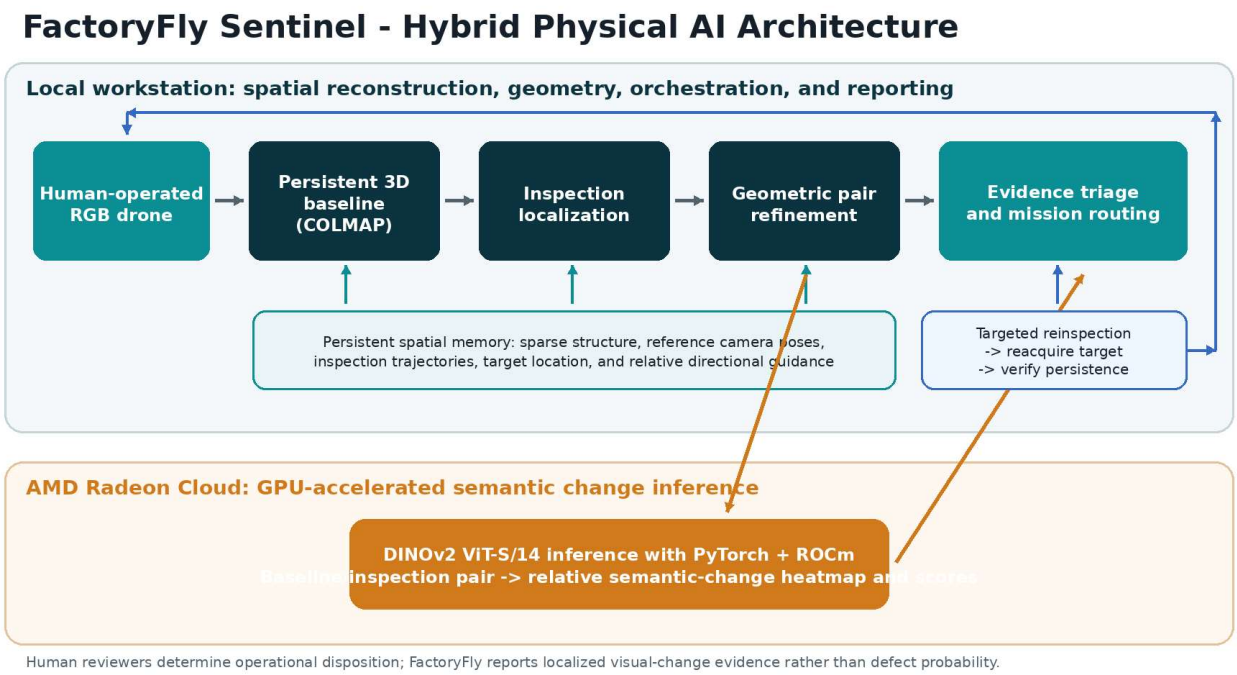


Figure 1. Hybrid architecture connecting local spatial processing with AMD Radeon Cloud inference.

The prototype uses a hybrid architecture. Spatial reconstruction, geometric processing, workflow orchestration, and the Streamlit interface run on a Windows workstation. The core semantic-change inference stage runs on an AMD Radeon GPU in Radeon Cloud using PyTorch and ROCm.

2.1 Thirteen-Step Workflow

Step	Stage	Purpose
1	Baseline Registration	Construct the persistent COLMAP baseline.
2	Baseline Spatial Memory	Store sparse structure, poses, trajectories, and relative coordinates.
3	Inspection Registration	Register a new inspection video and optional telemetry.
4	Spatial Localization	Add inspection poses to the fixed baseline coordinate system.
5	Localization Result	Review registration coverage and pose continuity.
6	Pair Refinement	Retrieve and geometrically refine comparable baseline views.
7	Pair Result	Select geometry-ready pairs for semantic analysis.
8	AMD Analysis	Execute DINOv2 inference on Radeon Cloud through ROCm.
9	AMD Result	Review heatmaps, scores, and machine-readable artifacts.
10	Change Triage	Clear stable evidence, confirm strong evidence, and route uncertainty.
11	Reinspection Mission	Generate a spatial target and return guidance.
12	Reinspection Analysis	Reacquire the target and test whether the change persists.
13	Final Report	Generate HTML, Markdown, and JSON evidence outputs.

2.2 Spatial Memory

The baseline coordinate system is preserved throughout the workflow. Inspection poses are added without bundle adjustment or triangulation, preventing the active baseline frame from drifting. Spatial outputs are expressed in reconstructed relative units, not calibrated metres.



Figure 2. Spatial mission map with baseline path, inspection path, approximate sparse structure, start point, and reinspection target.

The mission view provides approximate structure and directional context rather than a certified navigation or obstacle map.

2.3 Local and Remote Responsibilities

Local workstation	AMD Radeon Cloud
Video sampling; COLMAP reconstruction; inspection localization; pose retrieval; geometric refinement; triage; mission generation; Streamlit UI; reporting; remote orchestration.	ROCm-enabled PyTorch runtime; DINOv2 ViT-S/14 feature inference; semantic comparison; heatmap and score artifact generation.

3. Dataset

3.1 Data Source

The evaluation uses a self-built RGB video dataset recorded in a private indoor environment. It contains a baseline flight, an initial inspection flight, and a targeted reinspection flight.

No model training or fine-tuning was performed. The dataset was used only for reconstruction, localization, geometric evaluation, and inference-time change analysis.

3.2 Demonstration Dataset Statistics

Stage	Metric	Result
Baseline	Sampled frames	158
Baseline	Registered cameras	91
Baseline	Registration rate	57.59%
Baseline	Sparse 3D points	5,499
Inspection localization	Input frames	47
Inspection localization	Registered frames	18
Inspection localization	Registration rate	38.3%
Inspection localization	Longest continuous registered run	16 frames
Pair refinement	Candidate pairs evaluated	90
Pair refinement	AMD-ready pairs	12
Pair refinement	High-confidence pairs	8
Pair refinement	Median reprojection error	0.962 px

3.3 Data and Privacy Compliance

- All source footage was captured by the participant.
- The environment is a private indoor space.
- No faces, confidential documents, company systems, or employer intellectual property are required for the demonstration.
- Public submission assets can be reduced or redacted without changing the technical method.
- Third-party models and dependencies remain subject to their respective open-source licences.

4. Algorithms and Implementation

4.1 Baseline Reconstruction

FactoryFly samples frames from the baseline RGB video and uses COLMAP feature extraction, matching, and sparse reconstruction. Among multiple sparse models, the model with the strongest registration result is selected as the persistent baseline.

- Sparse 3D points
- Registered reference camera poses
- Reference frame identities
- Baseline trajectory
- Start position and relative orientation context

4.2 Inspection Localization

Inspection frames are registered against the existing baseline model through COLMAP image registration. The baseline geometry is held fixed, producing inspection poses in the same coordinate system.

- Registered-frame count and percentage

- Failed-frame list
- Longest continuous localized run
- Baseline and inspection pose exports
- Timeline and trajectory visualization

4.3 Geometric Pair Refinement

For each localized inspection frame, FactoryFly retrieves the top-K nearest baseline camera candidates. The demonstrated run used K = 5.

1. Mutual-ratio SIFT feature matching
2. Fundamental Matrix RANSAC
3. Homography RANSAC
4. Valid overlap estimation
5. Reprojection error
6. Combined refinement quality scoring

> **Pair-refinement result**
> 90 candidates evaluated: 3 Excellent, 5 Good, 4 Usable, and 6 Poor. Twelve non-poor pairs were prepared for AMD analysis, with a median reprojection error of 0.962 pixels.

Homography is treated as a planar approximation. Geometry quality gates semantic interpretation so that high visual difference alone is not mistaken for reliable change evidence.

4.4 AMD-Accelerated DINOv2 Change Analysis

Geometry-ready pairs are transferred to Radeon Cloud. A DINOv2 ViT-S/14 checkpoint executes through PyTorch and ROCm and produces a baseline image, inspection image, warped baseline, relative semantic-change heatmap, and score statistics.

> **Interpretation**
> The heatmap represents relative semantic difference within the valid geometric overlap. Warm colours are not calibrated defect probabilities or severity scores.

4.5 Evidence Triage

FactoryFly combines semantic evidence with geometric quality. The demonstrated policy used a confirmed-change p95 threshold of 0.62 and an uncertainty-routing threshold of 0.70. These are demonstration routing thresholds, not calibrated probabilities.

- Automatically cleared as stable
- Recorded as a confirmed visual change
- Routed to targeted reinspection

4.6 Targeted Reinspection and Change-Tolerant Reacquisition

A reinspection mission targets the baseline reference camera location associated with uncertain evidence. The mission view includes baseline and inspection trajectories, sparse context, relative X/Y/Z directions, and the target position.

Direct matching to the original baseline can fail precisely because the scene changed. FactoryFly therefore accepts either a usable direct baseline-to-reinspection match or a usable initial-inspection-to-reinspection match. In the latter case, baseline-to-initial and initial-to-reinspection homographies are composed, and the follow-up frame is cropped to the projected reference field of view.

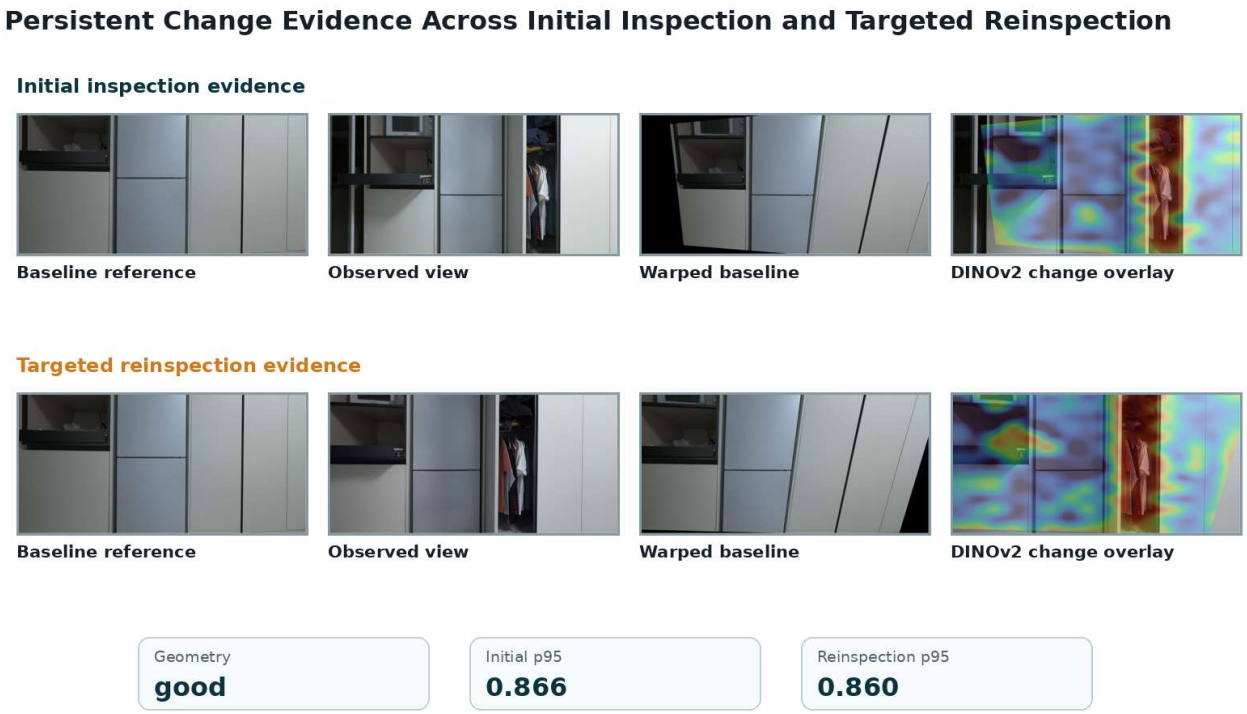


Figure 3. The same target is compared during the initial inspection and targeted reinspection. Good reacquisition geometry and nearly unchanged p95 evidence support persistence.

4.7 Reporting

- Summary counts
- Confirmed, cleared, and unresolved sections
- Baseline, inspection, warped, and heatmap evidence
- Reinspection geometry and p95 metrics
- Interactive 3D spatial context
- Human disposition and reviewer-note fields
- Explicit scope and interpretation guidance

The report can be opened without a running server because images and interactive data are embedded.

5. AMD Radeon GPU and ROCm Utilization

AMD Radeon GPU acceleration is used for the core AI perception stage. Spatial geometry and orchestration are intentionally separated from GPU inference, allowing the Radeon component to be independently tested and reproduced.

5.1 Execution Flow

1. Local geometry-ready pairs are packaged.
2. FactoryFly transfers the package to a Radeon Cloud workspace.
3. A ROCm-enabled Python environment loads the open-source DINOv2 model.
4. Feature inference and semantic comparison execute on the AMD Radeon GPU.
5. Result images, score tables, run metadata, and benchmarks are returned locally.
6. Local triage and reporting consume the returned artifacts.

5.2 Open-Source and Deployment Characteristics

- Core AI inference uses an open-source model.
- PyTorch executes through the ROCm software stack.
- No closed-source online API is used for core change inference.
- The remote inference package is reproducible through explicit environment, model, and dependency configuration.

- Local and remote responsibilities are isolated, making the Radeon inference component independently testable.

6. Results and Evaluation

6.1 End-to-End Run Summary

Metric	Result
Analyzed evidence entries	18
Stable observations automatically cleared	10
Confirmed visual findings	4
Targeted reinspections	1
Cleared after reinspection	0
Unresolved findings	0

6.2 Reinspection Result

The uncertain evidence cluster covering inspection frames 21-23 was linked to baseline frames 81-82 and routed to reinspection.

Reinspection metric	Result
Reacquisition geometry	Good
Initial p95	0.866
Reinspection p95	0.860
Final result	Persistent visual change confirmed

The p95 score changed by only 0.006, approximately 0.7% relative to the initial value. Together with good geometry, this supports the conclusion that the visual change remained observable after targeted follow-up.

6.3 What the Evaluation Demonstrates

- A later human-operated flight can be localized inside persistent 3D spatial memory.
- Geometric validation can filter unsuitable image comparisons.
- DINOv2 inference can run on AMD Radeon GPU through ROCm.
- Uncertain evidence can be converted into a physical re-observation task.
- The same target can be reacquired despite a large appearance change.
- The full evidence chain can be exported for human review.

The evaluation is a proof of concept rather than a statistically calibrated industrial benchmark.

7. Innovations and Key Technical Contributions

7.1 Persistent 3D Inspection Memory

Repeated inspections are connected through a fixed spatial coordinate system instead of being treated as isolated video files.

7.2 Geometry-Gated Semantic Comparison

Semantic difference is interpreted only when geometry is sufficiently usable, reducing false confidence caused by invalid overlap or parallax.

7.3 Uncertainty-Directed Physical Re-observation

The system converts insufficient evidence into an explicit reinspection mission rather than forcing a binary decision.

7.4 Change-Tolerant Reacquisition

An inspection-bridge homography reacquires targets that no longer match the original baseline directly.

7.5 Human-Centred Evidence Reporting

Observed visual change is separated from operational judgement, with evidence quality and limitations made explicit.

8. Final Deliverables

- FactoryFly Sentinel Streamlit application, version 7.3.13
- Thirteen-stage human-guided inspection workflow
- COLMAP-based baseline and localization pipeline
- Geometric pair-refinement pipeline
- AMD Radeon Cloud + ROCm DINOv2 inference workflow
- Evidence triage and targeted reinspection mission generation
- Change-tolerant reacquisition implementation
- Self-contained HTML evidence report
- JSON, CSV, image, and benchmark artifacts
- Demonstration video showing the complete workflow
- Source repository and reproducibility README

9. Limitations and Future Work

9.1 Current Limitations

- Sparse reconstruction uses relative scale rather than calibrated metric units.
- The mission map is approximate and is not a collision-free navigation planner.
- Homography is a planar approximation and can fail under large parallax or non-rigid motion.
- Localization depends on texture, overlap, blur, and lighting.
- Triage thresholds are demonstration policies, not calibrated probabilities.
- The current evaluation uses one private indoor environment and a limited number of flights.
- Drone operation and reinspection execution remain human-guided.
- The system reports visual change and does not infer defect class, severity, or safety impact.

9.2 Future Work

- Metric scaling through known references, depth, or visual-inertial fusion
- Autonomous or assisted camera-pose guidance during reinspection
- Multi-camera and thermal/RGB sensor fusion
- Larger industrial datasets and calibrated evaluation protocols
- Learned anomaly categories and maintenance-system integration
- Persistent multi-run inspection history and trend analysis
- On-device or edge Radeon deployment for lower-latency inference
- Robotic navigation and safety-constrained mission planning

10. Team Members and Contributions

Jaewon Lee - Solo Developer

- Problem definition and factory-inspection use-case design
- System architecture and thirteen-stage workflow design

- Private indoor data collection and test planning
- COLMAP baseline, localization, and spatial-memory integration
- Pair-refinement and geometric-quality logic
- Radeon Cloud, PyTorch, ROCm, and DINOv2 integration
- Evidence triage and targeted-reinspection design
- Change-tolerant reacquisition design and validation
- Streamlit user interface and self-contained HTML reporting
- End-to-end debugging, testing, documentation, and demonstration video

AI-assisted development tools were used under the participant's direction for code drafting, debugging support, and documentation. The participant performed the project-specific architecture decisions, data collection, integration, execution, verification, and final submission.

11. Conclusion

FactoryFly Sentinel demonstrates a practical Physical AI pattern for industrial inspection: persistent spatial memory, geometry-aware GPU perception, uncertainty-aware decision routing, targeted physical re-observation, and human-reviewed evidence reporting.

The prototype's central contribution is not simply detecting visual difference. It preserves where the evidence was observed, evaluates whether the comparison is geometrically credible, and initiates another physical observation only when necessary. The demonstrated run completed the full loop from baseline construction to persistent-change confirmation with no unresolved findings.

> **Core message**
> Find the change. Locate the evidence. Reinspect with purpose.

Technology Stack

Python, Streamlit, OpenCV, COLMAP, SIFT, RANSAC, PyTorch, ROCm, DINOv2 ViT-S/14, PowerShell, HTML, CSS, and JavaScript.